# Machine Learning for Agents

## Intelligent Systems II

Mário Antunes

Universidade de Aveiro

2026

# Part 1: The Foundations of Learning

# What is Machine Learning?

- **Definition**: A field of AI that provides systems the ability to automatically learn and improve from experience without being explicitly programmed.
- **The "Experience"** $(E)$: Data from the environment, historical logs, or sensor streams.
- **The "Task"** $(T)$: Predicting an outcome, classifying an object, or making a decision.
- **The "Performance"** $(P)$: A metric to measure how well the agent is doing (Accuracy, RMSE, Reward).
- **Shallow ML**: Traditional algorithms that typically require manual feature engineering (e.g., KNN, SVM, DT).

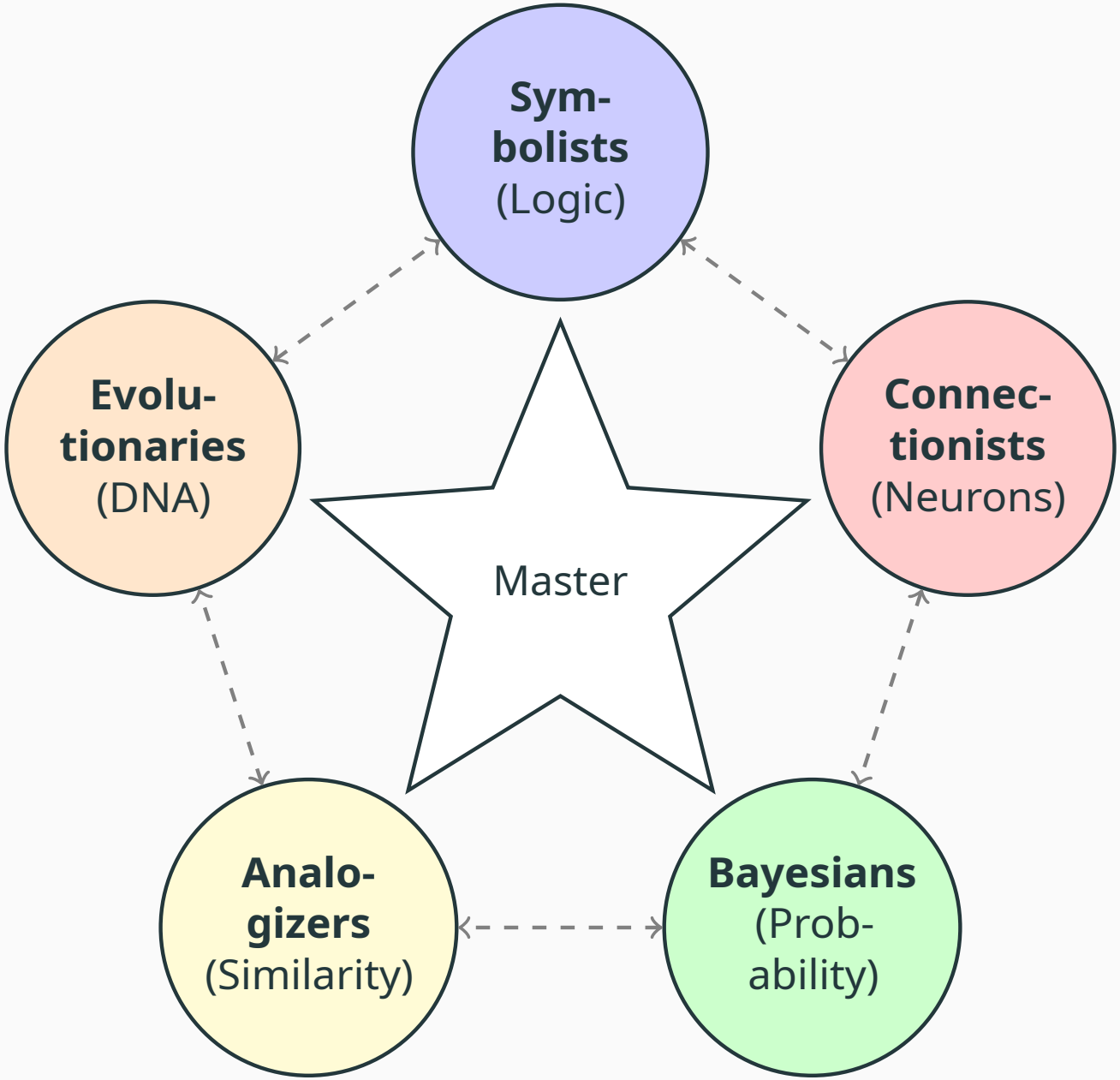
# The Master Algorithm: The Five Tribes i

*Based on Pedro Domingos' "The Master Algorithm", ML can be categorized into 5 tribes based on their core philosophy:*

| Tribe | Philosophy | Core Algorithm |
|---|---|---|
| **Symbolists** | Logic, Philosophy | Inverse Deduction |
| **Connectionists** | Neuroscience | Backpropagation |
| **Evolutionaries** | Evolutionary Biology | Genetic Programming |
| **Bayesians** | Statistics | Probabilistic Inference |
| **Analogizers** | Geometry, Similarity | Kernel Machines (SVM) |

*The "Master Algorithm" is the ultimate learner that would combine all these approaches.*

# The Master Algorithm: The Five Tribes ii



# Part 2: Learning Taxonomies

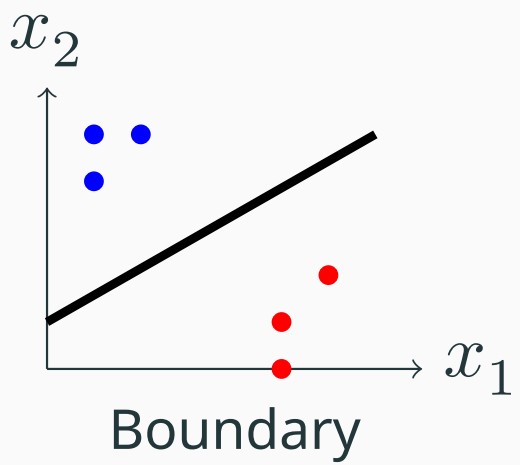
# Taxonomy I: Expected Response

- **Supervised Learning**: Learning from labeled data $(X, y)$. Mapping $f : X \to y$.
  - *Classification*: Discrete labels (e.g., “Safe” vs “Danger”).
  - *Regression*: Continuous values (e.g., distance to target).
- **Unsupervised Learning**: Learning from unlabeled data $(X)$. Structure discovery.
  - *Clustering*: Grouping similar environment states.
  - *Dimensionality Reduction*: Compressing complex sensor data.
- **Reinforcement Learning**: Learning via rewards/penalties.
  - *Focus*: Sequential decision making (next class topic).

# Taxonomy II: Discriminative vs Generative Models

**Discriminative Models**

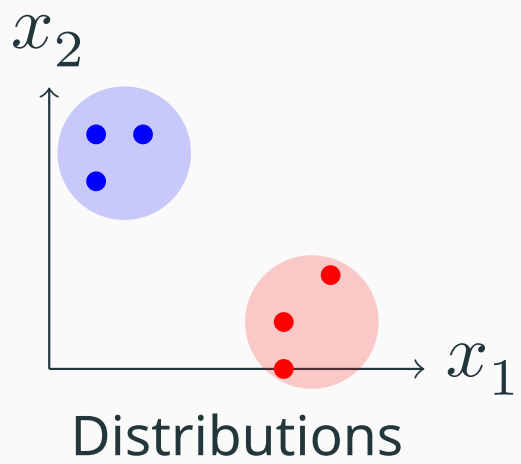
- Models the boundary: $P(y|X)$.
- Focused on classification tasks.
- "What is the difference between A and B?"



Boundary

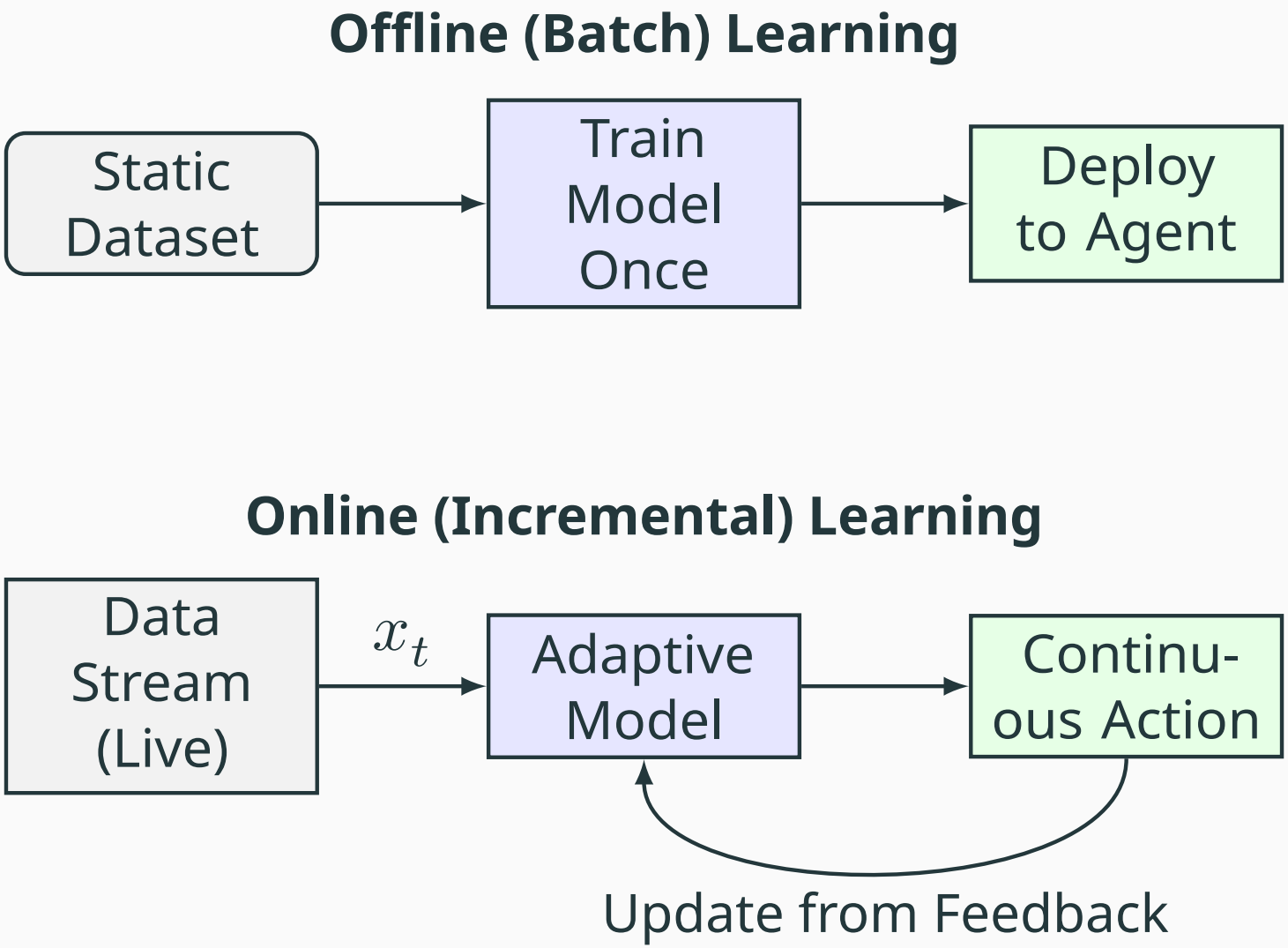
**Generative Models**

- Models the distribution: $P(X, y)$.
- Can generate new data points.
- "What does a typical A look like?"



Distributions

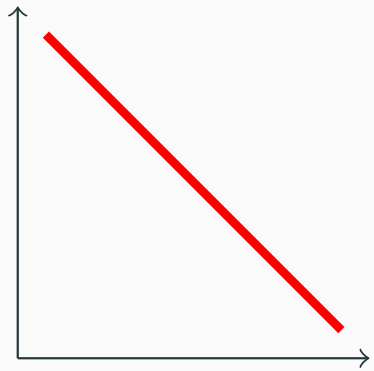
# Taxonomy III: Offline vs Online Learning



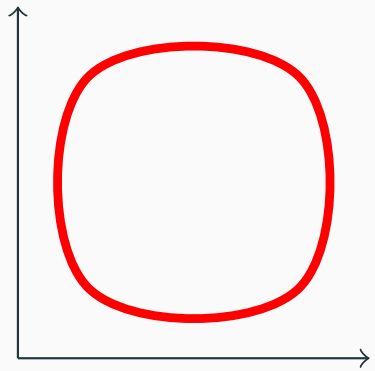
# Part 3: Decision Boundaries and Representation

# The Decision Boundary i

- **Definition**: A hypersurface that partitions the underlying vector space into sets, one for each class.
- **The Limit of Learning**: An agent's capability is strictly bounded by the complexity of the boundaries its model can represent.
- **Representational Capacity**: If the true relationship is a circle but the model only knows lines, it can never achieve 100% accuracy.

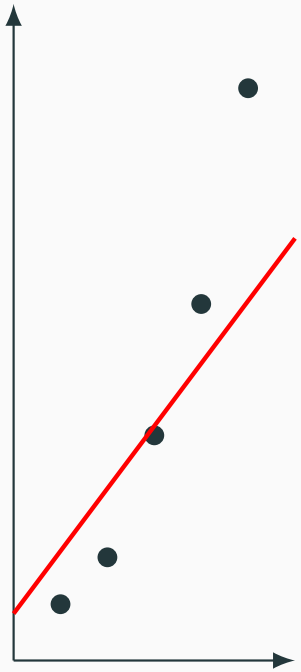


Linear (Simple)

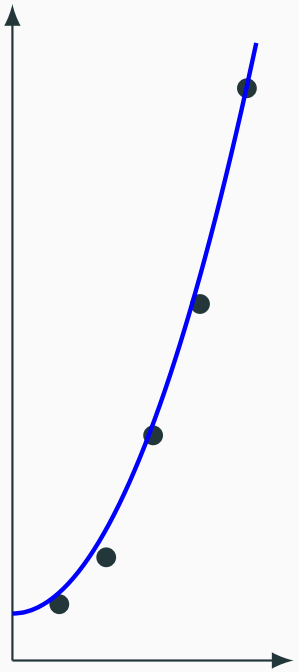


Non-linear (Complex)

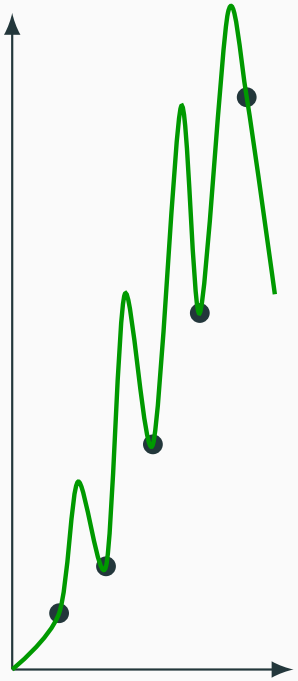
# The Decision Boundary ii: Model Fit



**Underfit**
(High Bias)



**Good Fit**
(Generalizable)

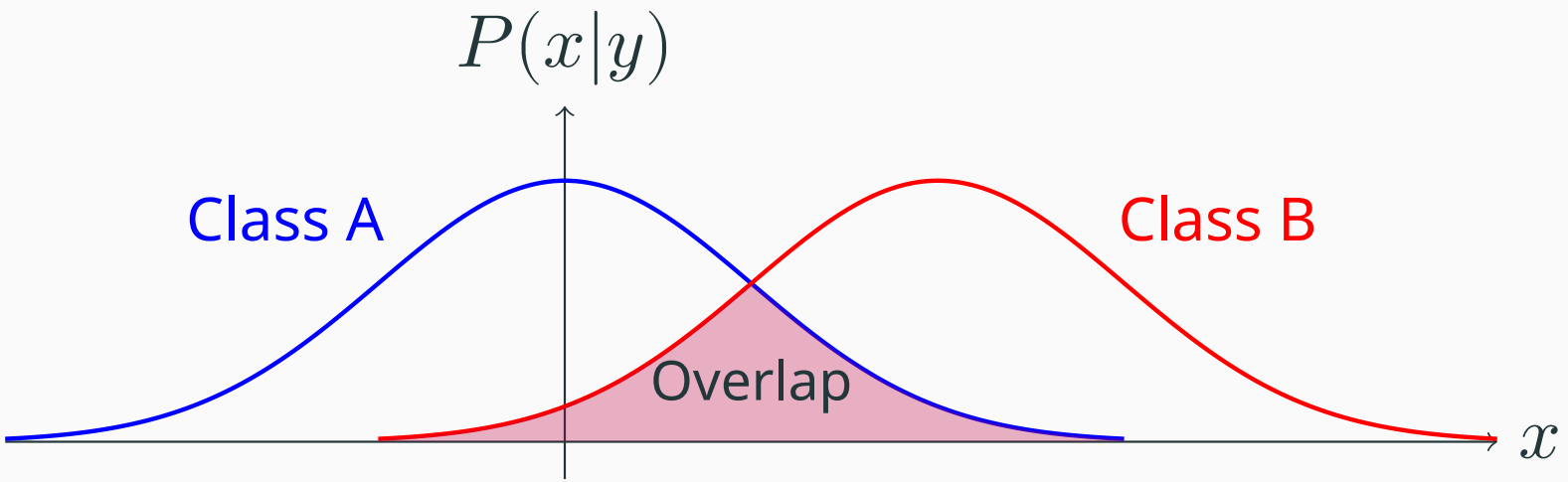


**Overfit**
(High Variance)

# Part 4: Fundamental Limitations

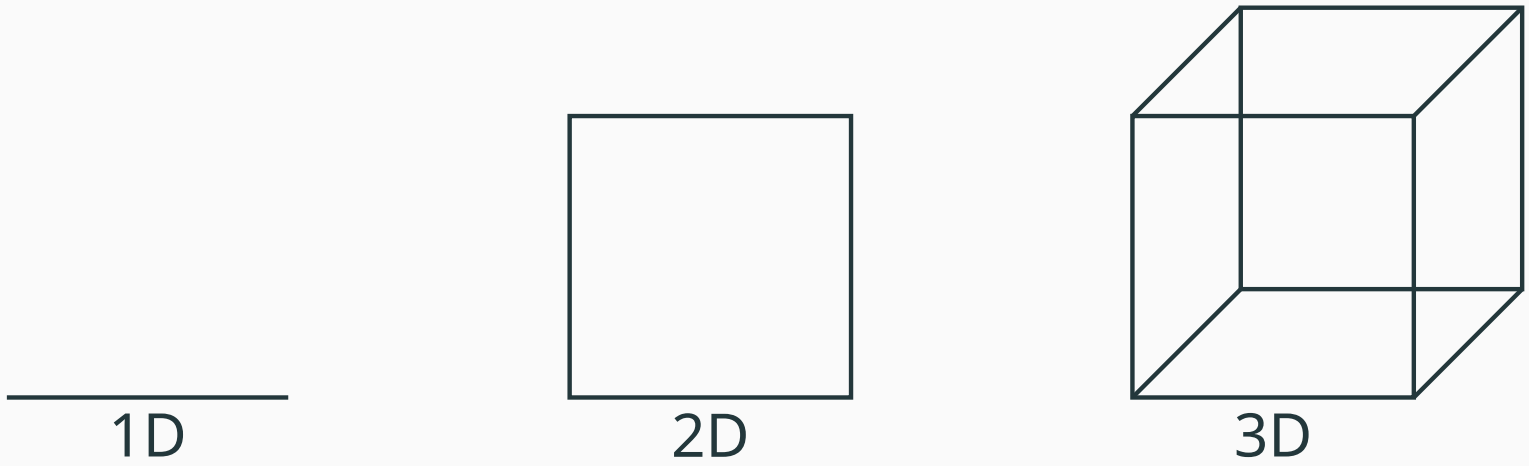
# Limitation I: Max Expected Performance

- In real-world agent environments, classes often overlap in feature space.
- **Bayes Error**: The minimum possible error rate.
- Overlap means that for some $X$, the same observation could belong to multiple classes.



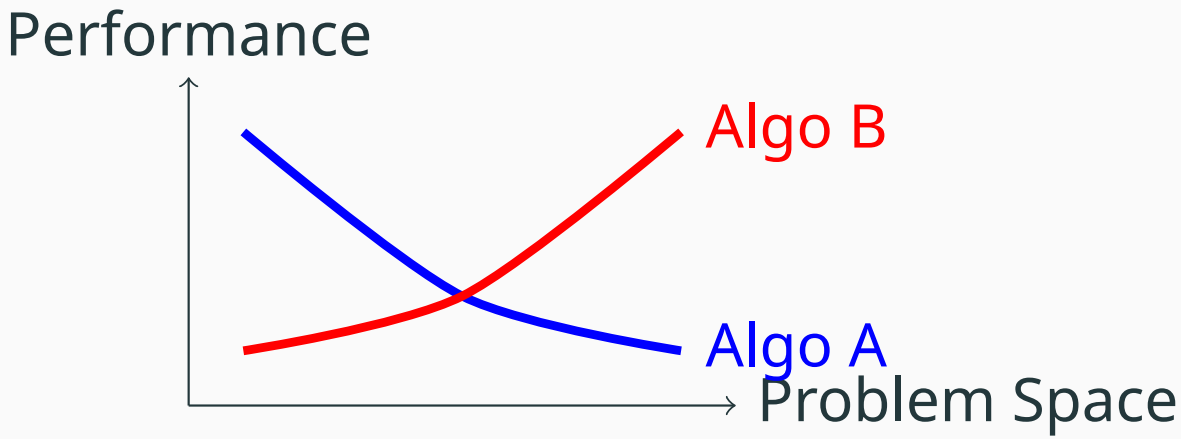
# Limitation II: The Curse of Dimensionality

- As the number of features increases, the volume of the space grows exponentially.
- Density of samples decreases, and points become isolated in corners.
- The “average distance” to neighbors increases significantly.



# Limitation III: No Free Lunch Theorem

- **Definition**: No single machine learning algorithm is universally better than any other across all possible problems.
- **Implication for Agents**: An agent optimized for a "Maze" environment may perform poorly in an "Open Field" without retraining or re-tuning.

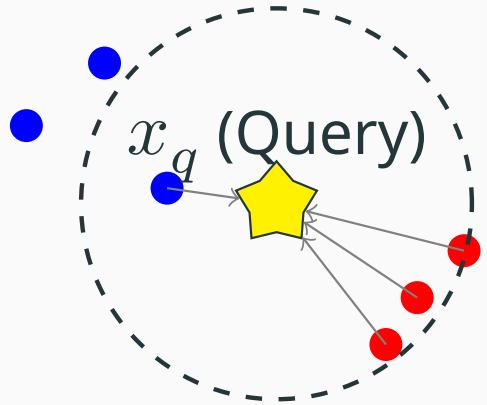



Inductive Bias determines success

# Part 5: Models

# KNN i: Intuition and Diagram

- **Core Idea**: “Similarity implies the same class”.
- **Lazy Learning**: No explicit training phase; just store the data.
- **Classification**: Query point takes the majority label of its $k$ closest neighbors.




$k = 3 \implies$ Red Wins

# KNN ii: Mathematical Formulation

- **Distance Metric**: Usually Euclidean distance in $d$-dimensional space.

$$d(x, y) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

- **Classification Rule**: Let $N_k(x)$ be the set of $k$ nearest training samples to query point $x$.

$$\hat{y} = \arg\max_{c \in \mathcal{C}} \sum_{x_i \in N_k(x)} I(y_i = c)$$

- **Regression Rule**:

$$\hat{y} = \frac{1}{k} \sum_{x_i \in N_k(x)} y_i$$

# KNN iii: Algorithms and Usage

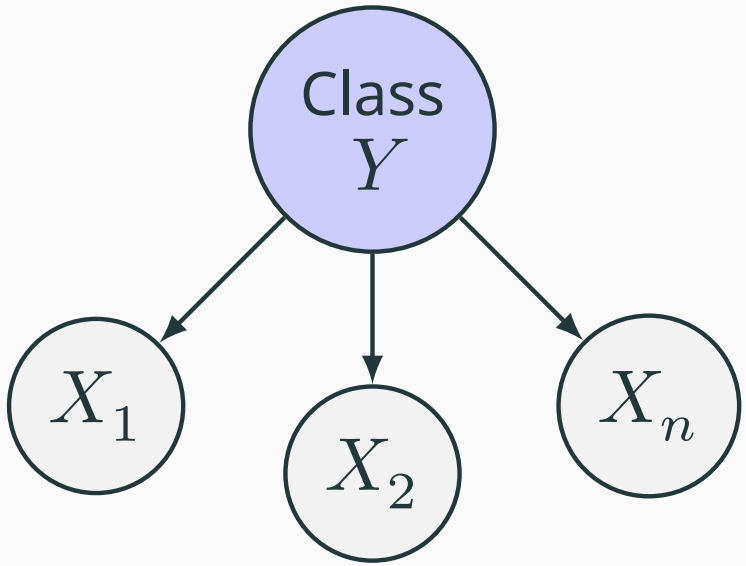
- **Algorithm**:
  1. **Training**: Store all labeled training pairs $(X, Y)$ in memory.
  2. **Inference**: For each query point $x_q$:
     - Calculate distance to all $N$ stored samples.
     - Sort samples and identify the $k$ nearest neighbors.
     - Apply majority vote (Classification) or mean (Regression).
- **Real-World Usage**:
  - **Recommendation Systems**: "Users like you also watched...".
  - **Agent Use**: Memory-based agents that recall the outcome of similar past states to decide current actions.

# KNN iv: Pros and Cons

- **Pros**:
  - Zero training time ($O(1)$).
  - Naturally handles multi-class problems.
  - Highly intuitive and non-parametric.
- **Cons**:
  - **Inference is slow**: $O(N \cdot d)$ per query.
  - **High Memory**: Must store the entire dataset.
  - **Curse of Dimensionality**: Performance degrades as dimensions increase.
  - Sensitive to irrelevant features and data scale.

# Naive Bayes i: Intuition and Diagram

- **Core Idea**: Uses Bayes' Theorem to find the probability of a class given the input features.
- **The "Naive" Part**: Assumes that every feature is completely independent of every other feature, given the class label.




$$P(X_1, X_2|Y) = P(X_1|Y)P(X_2|Y)$$

# Naive Bayes ii: Mathematical Formulation

- **Bayes' Theorem**:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

- **Independence Assumption**:

$$P(x_1, \dots, x_n|y) = \prod_{i=1}^{n} P(x_i|y)$$

- **Decision Rule**:

$$\hat{y} = \arg\max_y \left( P(y) \prod_{i=1}^{n} P(x_i|y) \right)$$

*We ignore $P(X)$ because it's constant for all classes.*

# Naive Bayes iii: Algorithms and Usage

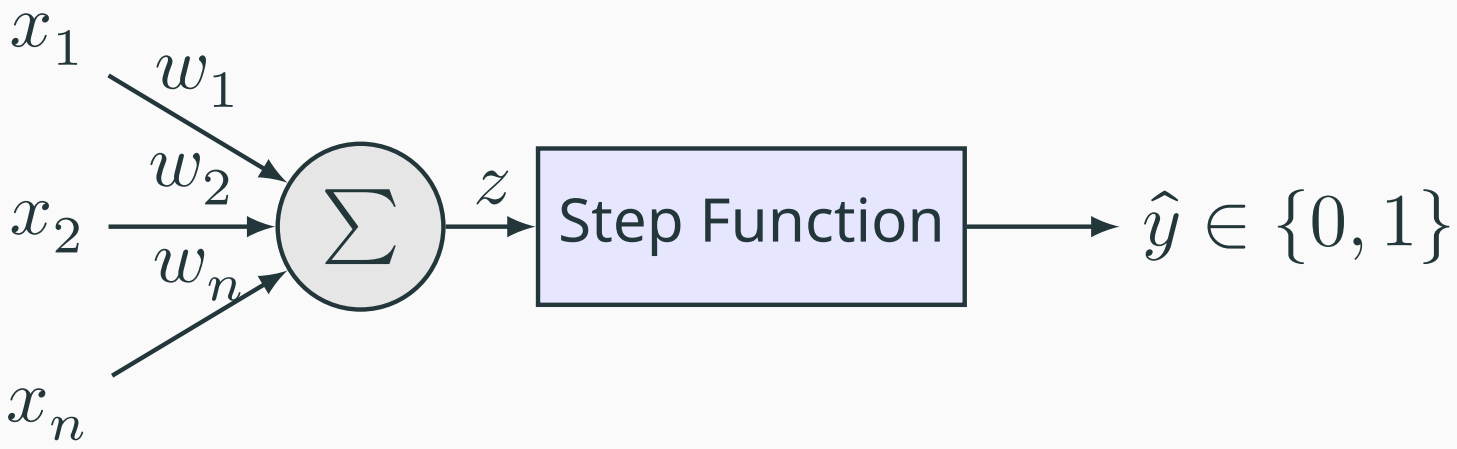
- **Algorithm**:
  1. **Training**: Count occurrences to estimate class priors $P(y)$ and conditional likelihoods $P(x_i|y)$.
  2. **Inference**: For new input $X_q$, multiply the learned probabilities and pick the class that maximizes the result.
- **Real-World Usage**:
  - **Spam Filtering**: Word probabilities determining if an email is junk.
  - **Agent Use**: Sensor Fusion. An agent treating different sensor readings as independent evidence for the state of the world.

# Naive Bayes iv: Pros and Cons

- **Pros**:
  - Extremely fast training and inference.
  - Works well with small datasets.
  - Scalable to very high-dimensional feature spaces.
- **Cons**:
  - **Independence Fallacy**: In real life, features are often correlated (e.g., in a car, speed and RPM are not independent).
  - **Zero Frequency Problem**: If a combination was never seen in training, the probability becomes zero (can be fixed with Laplace Smoothing).

# Perceptron i: Intuition and Diagram

- **Core Idea**: A mathematical model of a single biological neuron that performs binary classification.
- **Mechanism**: It sums weighted inputs and checks if the result exceeds a threshold.



# Perceptron ii: Mathematical Formulation

- **Input Combination**:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

- **Step Activation Function**:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

- **Update Rule (Learning)**:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$

*Where $\eta$ is the learning rate, $y$ is the true label, and $\hat{y}$ is the prediction.*

# Perceptron iii: Algorithms and Usage

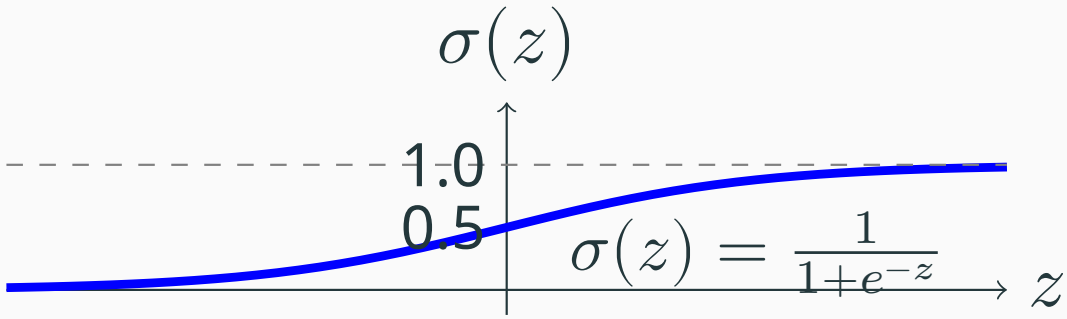
- **Algorithm**:
  1. Initialize weights **w** to small random values or zeros.
  2. For each training sample $(X, y)$:
     - Predict $\hat{y}$ using current weights.
     - If $\hat{y} \neq y$, update the weights using the error signal.
  3. Repeat until weights stabilize or error is minimized.
- **Real-World Usage**:
  - **Building Block**: Basis of modern Neural Networks and Deep Learning.
  - **Agent Use**: Simple reflex agents (e.g., move if average light intensity $>$ threshold).

# Perceptron iv: Pros and Cons

- **Pros**:
  - Extremely simple and computationally efficient.
  - Guaranteed to converge **if** the data is linearly separable.
- **Cons**:
  - **Linear Limitation**: Cannot solve problems that aren't separable by a line (like the XOR problem).
  - Can take a long time to converge on noisy data.
  - Hard output (0 or 1) provides no measure of confidence.

# Logistic Regression i: Intuition and Sigmoid

- **Core Idea**: Outputs a **probability** instead of a hard class label.
- **Activation**: Uses the Sigmoid function to map any value into the range $[0, 1]$.



# Logistic Regression ii: Mathematical Formulation

- **Probability Model**:

$$P(y = 1|x) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}}$$

- **Binary Classification**: We predict class 1 if $P(y = 1|x) \geq 0.5$.
- **Cost Function (Log Loss)**:

$$J(\mathbf{w}) = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

# Logistic Regression iii: Algorithms and Usage

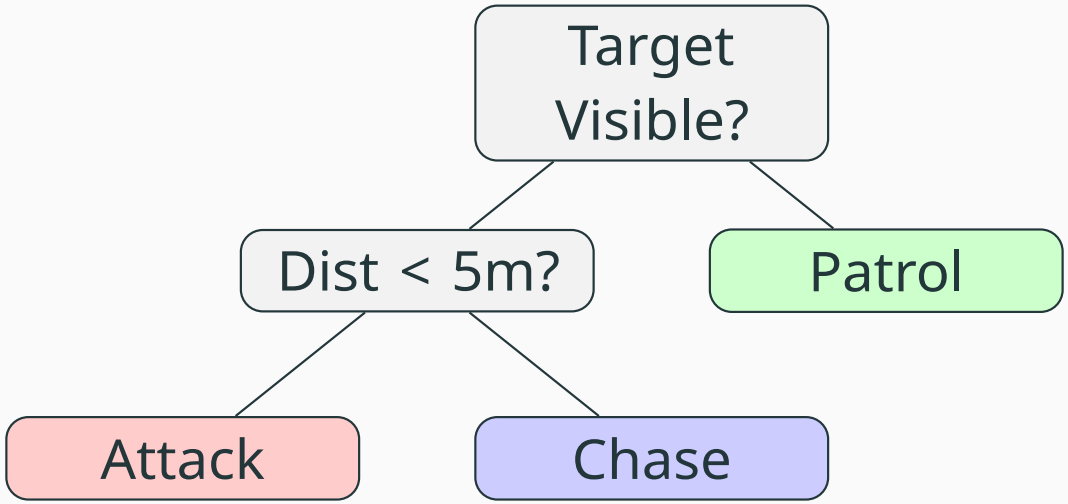
- **Algorithm**:
  1. Initialize weights **w**.
  2. Use **Gradient Descent** to find the weights that minimize the Log Loss.
  3. Iteratively update: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$.
- **Real-World Usage**:
  - **Medical Diagnosis**: Probability of having a disease based on symptoms.
  - **Ad-Click Prediction**: Likelihood a user will click on an ad.
  - **Agent Use**: Uncertainty-aware agents. Deciding to "Attack" if probability of winning $> 0.75$.

# Logistic Regression iv: Pros and Cons

- **Pros**:
  - Provides calibrated probabilities (confidence levels).
  - Robust to noise and less prone to overfitting than complex models.
  - Mathematically well-founded and interpretable.
- **Cons**:
  - Still fundamentally a **linear classifier**.
  - Assumes a linear relationship between features and the log-odds of the outcome.

# Decision Trees i: Intuition and Diagram

- **Core Idea**: Uses a tree-like model of decisions and their possible consequences.
- **Recursive Partitioning**: Splits the data into subsets based on feature values to maximize purity.



# Decision Trees ii: Mathematical Formulation

- **Entropy**: Measure of disorder in a set $S$.

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

- **Information Gain**: Difference in entropy before and after a split on attribute $A$.

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- **Gini Impurity**: Alternative split metric.

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

# Decision Trees iii: Algorithms and Usage

- **Algorithm (ID3/C4.5)**:
  1. Select the “best” attribute using Information Gain.
  2. Split the dataset into subsets based on attribute values.
  3. Repeat recursively for each branch until leaf nodes (pure classes) are reached or a stopping criterion is met.
- **Real-World Usage**:
  - **Credit Scoring**: Determining loan eligibility.
  - **Clinical Decision Support**: Diagnosis flowcharts.
  - **Agent Use**: Behavior Trees and human-interpretable policy generation.

# Decision Trees iv: Pros and Cons

- **Pros**:
  - Highly interpretable (White-box model).
  - Handles both numerical and categorical data.
  - Captures non-linear relationships.
- **Cons**:
  - **Overfitting**: Very prone to building overly complex trees (needs pruning).
  - **Instability**: Small changes in data can lead to a completely different tree structure.
  - High-variance model.

# Part 6: ML Examples for Agents

# Example: Hot/Cold Number Guess

- **Approach**: Use **Regression** to learn the mapping from (Guess, Feedback) to (Target).
- **Agent Logic**: Predicts the target location based on historical distance responses.

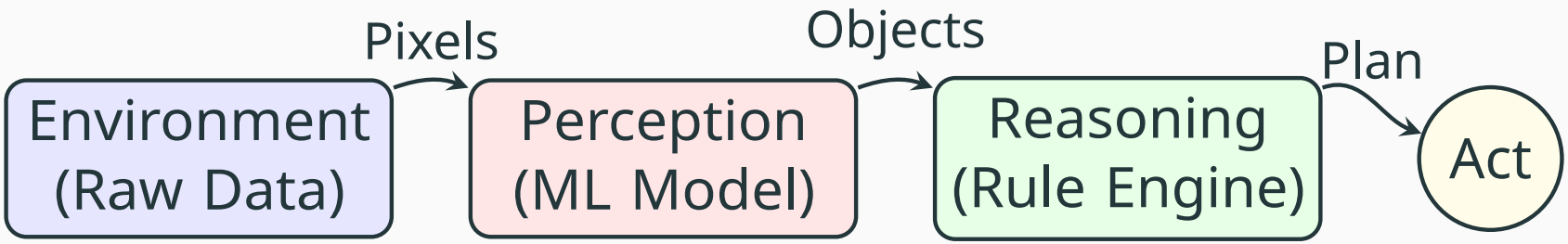
# Example: Tic-Tac-Toe

- **Approach**: Use a **Classifier** (e.g., Perceptron or DT) to evaluate the "strength" of a board state.
- **Agent Strategy**: Evaluates all next moves and picks the one with the highest Win probability.

# Part 7: Hybrid Architectures

# Perception vs. Reasoning

- **ML Layer**: Excellent at handling noisy, raw data (pixels, sound, lidar) to identify abstract objects.
- **Logic Layer**: Excellent at high-level planning, safety rules, and strategic reasoning.



# Hybrid Model Examples

- **Self-Driving Car**:
  - **ML**: Detects “Stop Sign” from camera feed.
  - **Logic**: If Sign detected AND Dist $<$ 5m, THEN activate Brake.
- **NPC Game Agent**:
  - **ML**: Predicts the player’s next move based on past behavior.
  - **Logic**: Uses A* Search to plot an intercept path.
- **Medical Assistant**:
  - **ML**: Identifies potential anomalies in medical images.
  - **Logic**: Cross-references findings with clinical guidelines and patient history.
- **Smart Home**:
  - **ML**: Predicts when the user will arrive based on phone GPS patterns.
  - **Logic**: Optimizes thermostat schedule based on energy cost rules.

# Example: Robot Navigation

- **Approach**: Map LIDAR sensor vectors directly to steering actions using a learned model.
- **Advantage**: Learns complex geometric patterns (corners, dead ends) from demonstrations.

# Conclusion

# Summary

- **ML Agents** learn from experience $(E)$ to perform tasks $(T)$ better over time $(P)$.
- **Classical Models** (KNN, NB, Perceptron, Logistic, DT) offer different trade-offs in speed, interpretability, and complexity.
- **Limitations**: Overlap, Dimensionality, and Inductive Bias define the boundaries of what is learnable.
- **Hybrid Architectures** combine the perception of ML with the strategic reliability of Logic.